Permissive Cross-Origin Access to the Prediction API and the Move to a Background-Mediated Origin

Divyam Saluja
GitHub: salujadivyam
LinkedIn: salujadivyam

Abstract, The SpoilFree prediction API accepted requests from any web origin. Restricting access to known origins was difficult while the content script called the API directly, because such requests carry the origin of the page being browsed. This report documents the weakness and the redesign that routes prediction requests through the extension background service worker, together with a server-side origin allow-list. Tests with a stubbed model returned 403 for a foreign origin and 200 for the configured origin. The allow-list remains unset until the published extension ID is known, and no exploitation was observed.

Index Terms*CORS, origin validation, service worker, API security, Chrome extension.*

I. INTRODUCTION

Cross-Origin Resource Sharing (CORS) is a browser mechanism through which a server indicates which origins may read a response to a cross-origin request. The browser compares the request origin with the server's CORS response headers; a wildcard policy permits browser scripts from any site to read eligible responses [1], [2]. CORS is therefore an important browser boundary, but it should not be treated as user authentication or as a complete defence against direct API clients.

SpoilFree uses a prediction API to classify text that may contain spoilers. The original configuration allowed every web origin. The issue was not simply a permissive header: the extension's request path made a narrow origin policy difficult to apply. This report describes the observed design problem, the background-mediated request flow and the checks performed during development.

II. WEAKNESS DESCRIPTION

The FastAPI service was configured with a wildcard origin list. The content script made prediction requests directly from the page context. A content-script request of this type carries the origin of the page being visited, which changes as the user navigates between websites [4]. As a result, the API could not use one stable extension origin to distinguish the intended extension from ordinary websites.

This is an origin-validation weakness: the server's policy did not restrict browser-readable responses to the intended caller [3]. A site able to cause a browser to make a request could potentially attempt to access the API response, subject to browser request and response rules. The investigation did not observe an attack, so the report records an exposure in configuration rather than a confirmed exploitation event.

The distinction matters because CORS is enforced by browsers. A non-browser client can construct an Origin header of its own choosing. Consequently, a server-side origin check can reduce casual browser-based cross-origin access, but it cannot independently prove that a request came from the genuine extension.

III. REDESIGN AND IMPLEMENTED MEASURES

A. Background-mediated requests. The content script now sends a message to the extension background service worker instead of calling the prediction API directly. The worker checks that the sender ID matches the extension ID and that the message comes from a tab. It accepts only a string payload of at most 2,000 characters, aborts the network request after eight seconds and sends no credentials [4], [5]. These checks reduce the amount of data accepted by the privileged request path and limit how long a stalled request can remain active.

B. Server-side allow-list. The API reads its allowed origins from an environment variable. When the variable is set, a request carrying a different Origin header receives status 403. A request with the configured origin is permitted. This makes the expected extension origin explicit at the server boundary and keeps deployment configuration separate from source code.

C. Deployment dependency. The current configuration preserves the previous wildcard behaviour when the environment variable is unset. This was retained temporarily so the service would continue to work before the published Chrome Web Store extension ID was available. It is also the main remaining configuration risk: the restriction is not active until the variable is configured. The deployment should fail closed or otherwise make this condition visible before public release.

IV. VERIFICATION AND EVIDENCE

Using a stubbed model and a test client, a request with a foreign Origin returned 403 and a request with the configured Origin returned 200. The extension message flow was exercised on a fake page with a stubbed message reply. These checks provide evidence that the server's conditional origin policy and the tested message path behaved as intended in the authoring environment.

The background worker was not run in a real Chrome profile during authoring. The available evidence therefore does not establish end-to-end behaviour in a packaged extension, compatibility across Chrome versions or protection against direct non-browser clients. A successful response for the configured origin also does not demonstrate that the API is authenticated.

V. RESIDUAL RISK AND RECOMMENDED CHECKS

The Origin header is not a credential and can be forged by non-browser clients. The allow-list should be considered a browser-facing restriction, not an authentication mechanism. Request size limits, rate limits and server-side validation remain necessary because a client can contact the API outside the browser-origin model.

Before release, set the exact published extension origin in the deployment environment and confirm that startup or health checks make an unset value visible. Then load the packaged extension in a clean Chrome profile and verify the content script to worker message, the sender checks, the timeout path and the API response. Test both a permitted origin and a foreign origin, and confirm that no credentials are attached.

These are recommended follow-up checks, not tests claimed as completed.

VI. CONCLUSION

The wildcard CORS configuration left the prediction API broadly readable from browser contexts. Moving requests to the extension background worker gives the API a stable extension-origin path and allows the worker to validate sender identity and bound the payload and request duration. The server-side allow-list was tested with a stubbed model, but remains inactive while its environment variable is unset. Configuring the published extension ID and performing a real Chrome end-to-end test are the most important remaining steps.

REFERENCES

[1] Mozilla, “Cross-Origin Resource Sharing (CORS),” MDN Web Docs. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

[2] A. Barth, “The Web Origin Concept,” IETF, RFC 6454, Dec. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6454

[3] MITRE, “CWE-346: Origin Validation Error.” Accessed: Oct. 28, 2026. [Online]. Available: https://cwe.mitre.org/data/definitions/346.html

[4] Google, “Cross-origin network requests,” Chrome for Developers. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/concepts/network-requests

[5] Google, “Message passing,” Chrome for Developers. Accessed: Oct. 28, 2026. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/concepts/messaging